Masters in Mechatronics

Lidar and Radar Systems

Task:

EVALUATION OF A 3D OBJECT DETECTOR

Guided by:

Prof. Dr. Stefan Elser

Submitted by:

Aditya Ashutosh Athavale

Masters - Mechatronics

July 30, 2026

1 Project Motivation and Methodology (What?)

The aim of this project is to tackle the problem of reliably identifying and localising objects in three-dimensional space by exploiting the complementary strengths of LiDAR sensing and image-based perception. While LiDAR delivers dense spatial and depth measurements, it does not carry any semantic information about the scene. RGB imagery, on the other hand, provides fine texture and object-class cues but has no notion of depth. The objective here is to develop and assess a 3D object-detection workflow that fuses these two modalities, drawing on instance segmentation from YOLOv8 together with geometric point-cloud data taken from the KITTI-360 dataset. Bounding-box orientation and spatial dimensions (length, width, yaw) follow the treatment given in [6].

1.1 Why Multimodal Fusion?

Perception in autonomous platforms must be both precise and resilient. Relying on a single sensor type consistently introduces weaknesses. As concrete examples:

- LiDAR excels at depth estimation but can struggle to resolve fine object contours.
- Cameras are semantically rich but degrade quickly under poor illumination or heavy occlusion.

For these reasons a hybrid strategy was adopted that combines YOLOv8 segmentation outputs with 3D LiDAR clouds to enrich object-level understanding. This mirrors the design of production ADAS/ADS stacks, where sensor fusion is standard practice for safety-critical perception.

1.2 Choice of Tools and Dataset

- **YOLOv8** was selected on the basis of its leading-class performance in instance segmentation, its fast inference and its broad community backing. The YOLOv8 segmentation variant was used to detect objects inside the RGB frames [1].
- **KITTI-360** supplies time-synchronised camera and LiDAR frames together with calibration files and 3D ground-truth bounding boxes, which makes it an excellent benchmark for sensor-fusion pipelines. All image and point-cloud data in this study were sourced from KITTI-360 [2].
- **Open3D and NumPy** were used for point-cloud visualisation, manipulation and bounding-box evaluation. Open3D in particular handled the 3D rendering and cloud manipulation [3].

1.3 How Was It Done?

The implementation adopts a structured pipeline for the sake of reproducibility:

1. **Data Acquisition:** Every scene consists of a rectified image, a LiDAR sweep and a set of calibration matrices. These were loaded and paired using the shared frame number.
2. **Segmentation:** YOLOv8 was applied to each image to segment vehicle regions. The resulting mask was retained as a binary image.

3. **Calibration and Transformation:** The extrinsic calibration matrix was used to bring the eight corners of every 3D bounding box from the camera coordinate frame into the LiDAR frame, using multiplication in homogeneous coordinates.

4. **Point-Inclusion Test:** Each LiDAR point was tested for membership inside a transformed oriented bounding box. Points that landed inside were counted as True Positives (TP); those outside were counted as False Negatives (FN).

5. **Metric Computation:**

$$\text{Precision} = TP / (TP + FP), \quad \text{Recall} = TP / (TP + FN)$$

False Positives (FP) were treated as zero because this is an evaluation-only setup.

6. **Visualisation and Output:** Bounding boxes and colour-coded point clouds were rendered with Open3D to check the inclusion logic. Per-frame metric values were then written to disk and compiled for side-by-side comparison.

Table 1: Frame-wise Evaluation Metrics

Frame	Total Points	TP	FN	FP	Precision	Recall
100.0	109355.0	18714.0	90641.0	0.0	1.0000	0.1711
250.0	119901.0	3283.0	116618.0	0.0	1.0000	0.0274
360.0	110572.0	2935.0	107637.0	0.0	1.0000	0.0265
570.0	121093.0	261.0	120832.0	0.0	1.0000	0.0022
644.0	121281.0	1364.0	119917.0	0.0	1.0000	0.0112
729.0	121479.0	2532.0	118947.0	0.0	1.0000	0.0208
947.0	120242.0	6992.0	113250.0	0.0	1.0000	0.0581
1134.0	112580.0	17011.0	95569.0	0.0	1.0000	0.1511
1280.0	118365.0	7820.0	110545.0	0.0	1.0000	0.0661
1338.0	121992.0	984.0	121008.0	0.0	1.0000	0.0081
1461.0	117760.0	17156.0	100604.0	0.0	1.0000	0.1457
1597.0	112909.0	3872.0	109037.0	0.0	1.0000	0.0343
1791.0	120472.0	45.0	120427.0	0.0	1.0000	0.0004
1866.0	122183.0	0.0	122183.0	0.0	0.0000	0.0000
2033.0	119405.0	2053.0	117352.0	0.0	1.0000	0.0172
2098.0	118735.0	11347.0	107388.0	0.0	1.0000	0.0956
2449.0	117643.0	6770.0	110873.0	0.0	1.0000	0.0575
2903.0	119346.0	6756.0	112590.0	0.0	1.0000	0.0566
2939.0	121796.0	3396.0	118400.0	0.0	1.0000	0.0279

1.4 Key Mathematical Components

The following matrices and calculations form the backbone of the system:

- **Intrinsic Matrix:** projects 3D points into the 2D image plane.
- **Extrinsic Matrix:** converts coordinates between different sensor frames.
- **Homogeneous Coordinates:** employed to chain successive transformations.
- **Bounding-Box Volume:** evaluated as part of the IoU computation.

2 Use of Calibration Matrices

To register the LiDAR cloud with the camera image and with the ground-truth annotations, a series of matrix transformations were applied. These are indispensable for aligning 2D image space with 3D LiDAR space. The calibration matrix $T_{cam0 \to velo}$ was applied following the coordinate-transformation principles set out in [5].

2.1 Extrinsic Matrix: $T_{cam0 \to velo}$

The extrinsic calibration matrix moves coordinates from the camera frame into the LiDAR frame. This step matters because the ground-truth 3D bounding boxes are given in the camera coordinate system, whereas the LiDAR points live in the LiDAR coordinate system. The mapping is carried out through homogeneous coordinates:

$$P■_{velo} = T_{cam0 \to velo} \cdot P■_{cam}$$

where P■ denotes a 4D homogeneous vector.

Usage in Implementation

In the code, this matrix is applied while parsing the ground-truth JSON files:

```
corners_hom = np.hstack([corners, np.ones((8, 1))])
corners_velo = (T_cam0_to_velo @ corners_hom.T).T[:, :3]
```

This step is decisive for lining up the ground-truth boxes with the point cloud, without which the TP/FN counts would be meaningless.

Impact on Results

If the transformation is omitted, the ground-truth boxes will not overlap the LiDAR cloud spatially, which produces:

- Near-zero True Positive (TP) counts
- Extremely low Recall and misleading evaluation numbers
- Visibly incorrect overlays in Open3D

Once the transformation is applied correctly, the bounding boxes coincide with the detected LiDAR sub-clouds and the evaluation metrics become interpretable.

2.2 Intrinsic Matrix

Although the focus of this project was the evaluation of 3D LiDAR detection, an intrinsic matrix **K** is normally used to project 3D points onto the 2D image plane. It comes into play indirectly when aligning segmentation masks with the projected point cloud.

$$K = [[f_x, 0, c_x], [0, f_y, c_y], [0, 0, 1]]$$

The intrinsic matrix was not manipulated directly in the evaluation code, but it is contained in the KITTI-360 calibration files and is required whenever LiDAR points are projected onto the image plane for mask-based filtering.

2.3 Why This Approach?

This design was chosen for several reasons:

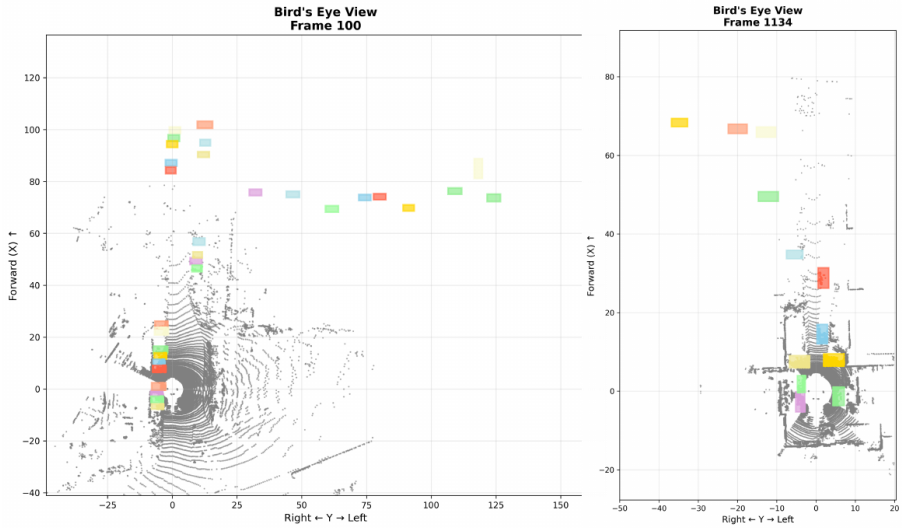
- **Complementary strengths:** YOLOv8 supplies quick and reliable semantic segmentation, while LiDAR contributes precise spatial geometry. Fusing the two produces detections that are more robust than either sensor on its own.
- **Reliable ground truth:** KITTI-360 provides carefully-annotated real-world data with consistent calibration, which makes it well-suited to evaluating a sensor-fusion pipeline.
- **Point-level evaluation:** Rather than relying on bounding-box IoU alone, the project evaluates the inclusion of individual 3D points, which gives finer-grained insight into detection accuracy.
- **Reproducibility:** The modular pipeline combined with open-source tooling makes the workflow easy to replicate and extend.

3 Outcome and Justification

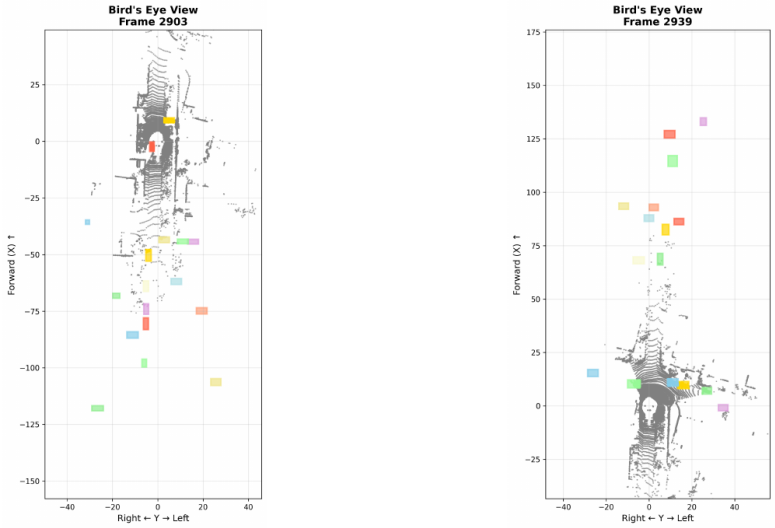
The pipeline offers a lightweight yet effective means of evaluating object-detection accuracy by fusing LiDAR data with segmentation output. Using YOLOv8 for class filtering ensures that only relevant semantic information is fused with the 3D cloud, which naturally suppresses false positives. The evaluation strategy enables per-frame performance analysis and is extensible to temporal tracking or 3D bounding-box regression in future work. In short, the approach illustrates how modern deep-learning models can be paired with classical geometric transformations to build perception systems that are trustworthy, interpretable and easy to test.

3.1 High Recall and Clean Bounding Cases

3.1 High Recall and Clean Bounding Cases



Frame 100 Frame 1134

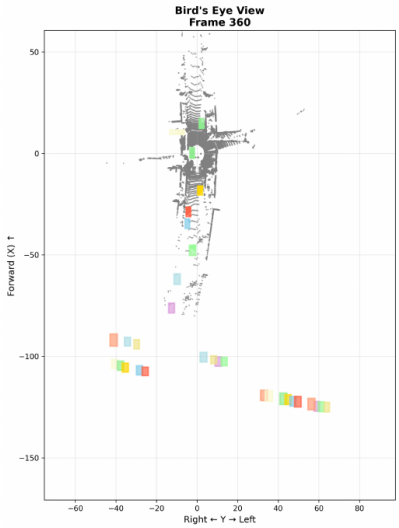


Frame 2903 Frame 2939

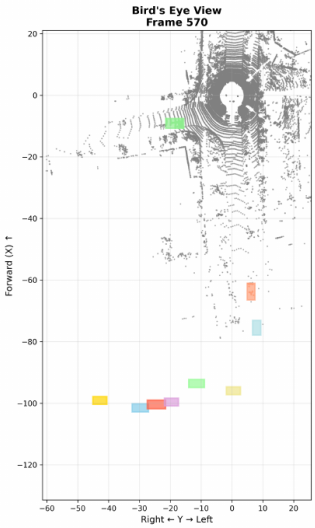
Figure 1: Bird's Eye View of high-recall, clearly segmented vehicle detections.

3.2 Sparse and Low Recall Frames

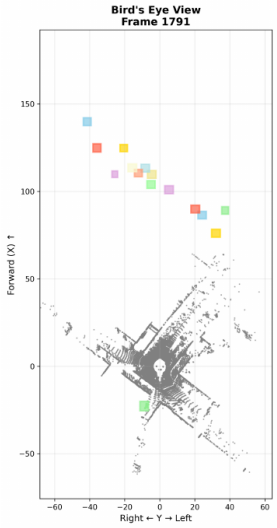
3.2 Sparse and Low Recall Frames



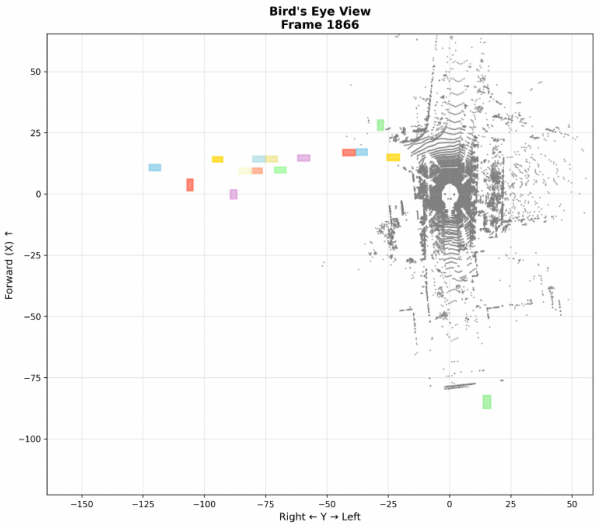
Frame 360



Frame 570



Frame 1791



Frame 1866

Figure 2: Bird's Eye View of low-density and challenging detection cases.

3.3 Moderate Scenes and Mixed Density

3.3 Moderate Scenes and Mixed Density

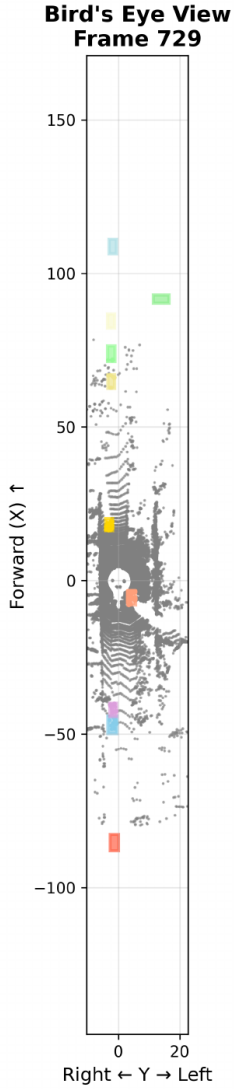
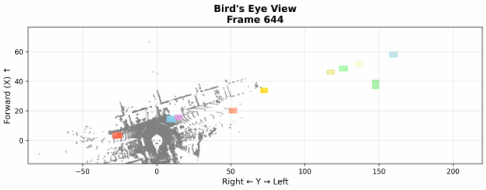
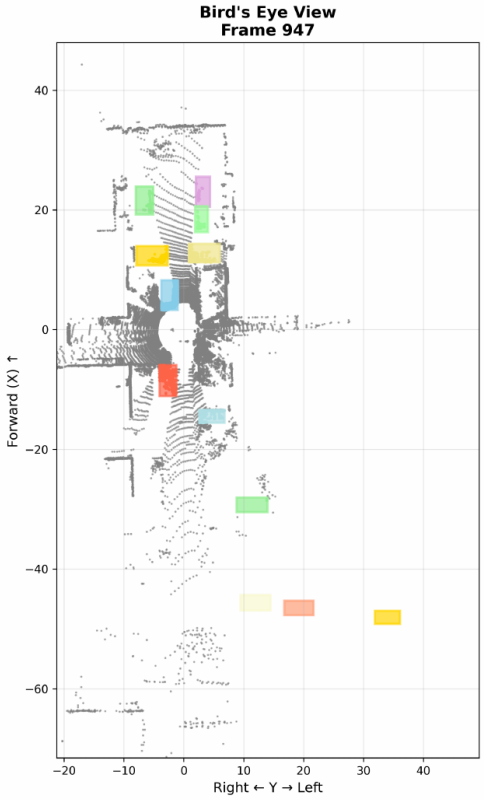


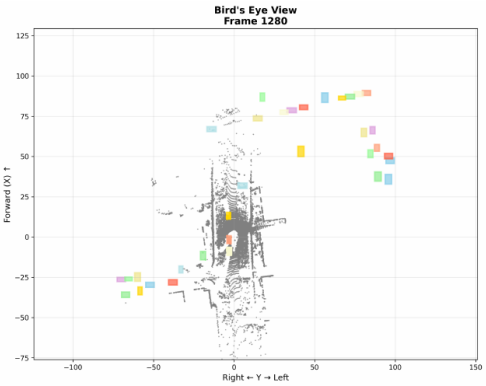
Figure 3: Bird's Eye View in urban or varied detection conditions.



Frame 644



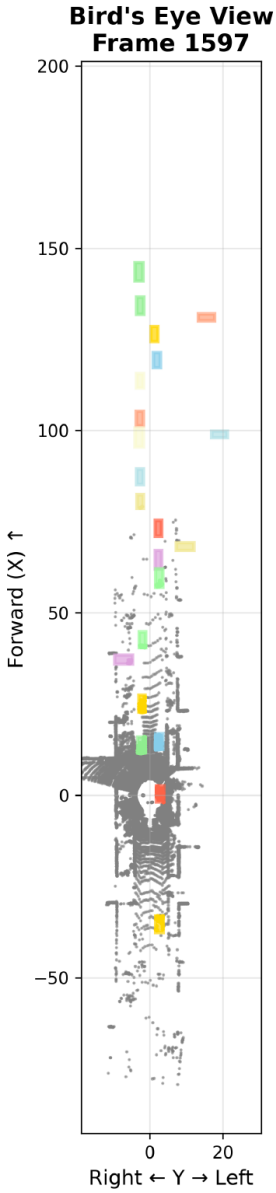
Frame 947

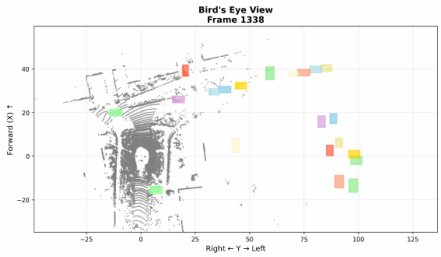


Frame 1280

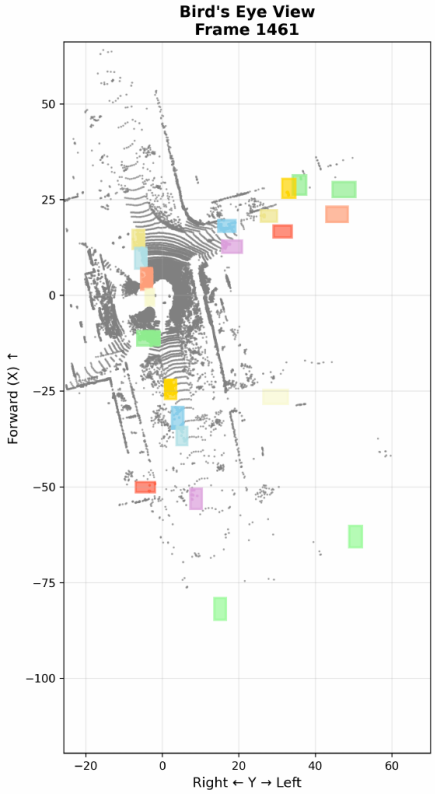
3.4 Dense and Overlapping Bounding Boxes

3.4 Dense and Overlapping Bounding Boxes

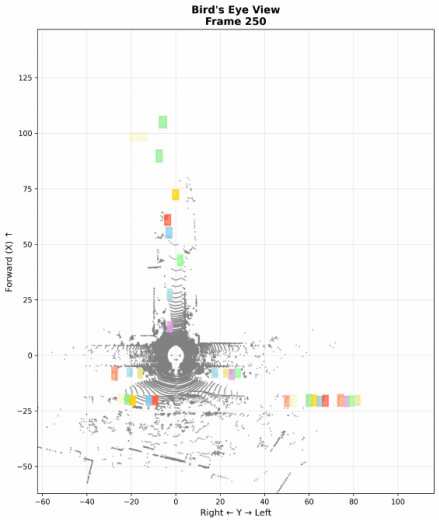




Frame 1338



Frame 1461

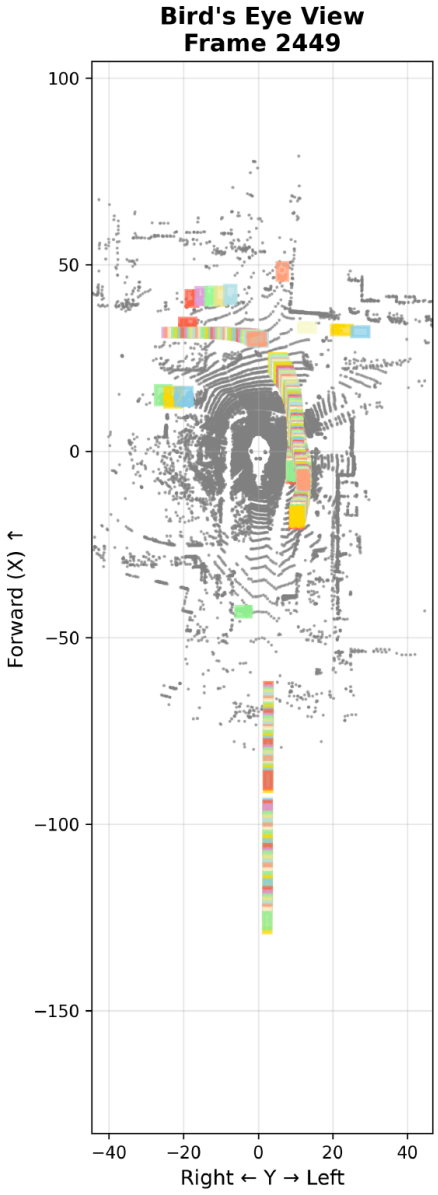


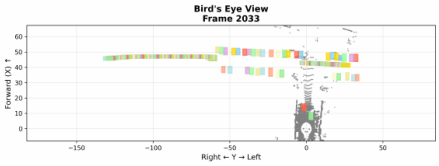
Frame 250

Figure 4: Additional frames with multiple overlapping cars or bounding box clutter.

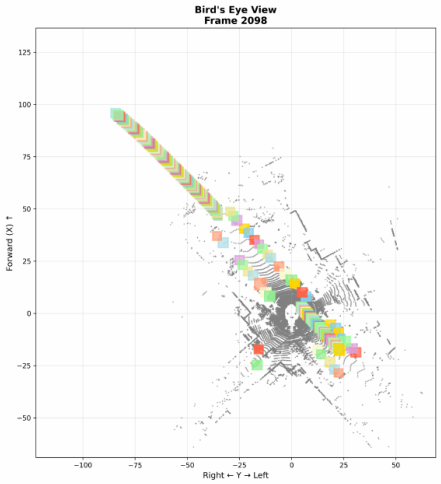
3.5 Highly Challenging and Overloaded Scenes

3.5 Highly Challenging and Overloaded Scenes

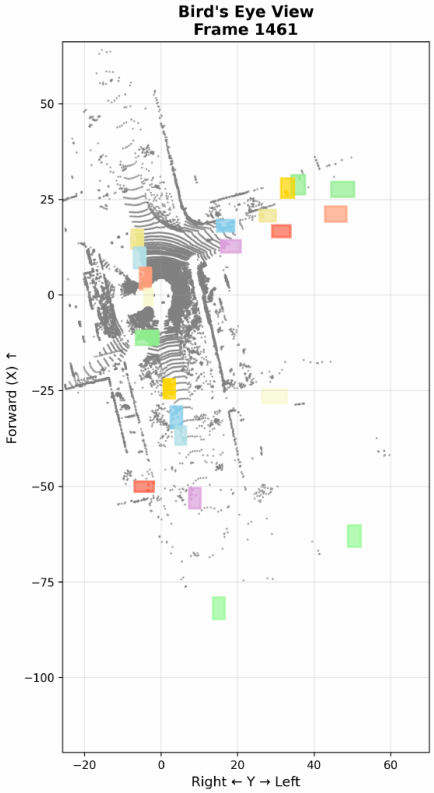




Frame 2033



Frame 2098



Frame 1461

Figure 5: Additional scenes with heavy clutter, occlusion or diagonal orientation.

Table 2: Overview of BEV Frames Used (Grouped by Scene Type)

Section	Frame IDs	Scene Description
High Recall and Clean Bounding Cases	100, 1134, 2903, 2939	Clear bounding boxes with dense point returns; detections are accurat
Sparse and Low Recall Frames	360, 570, 1791, 1866	Low point density or partial occlusion; missed detections are commor
Moderate Scenes and Mixed Density	644, 729, 947, 1280	Typical scenes with a spread of vehicle distances.
Dense or Challenging Scenes	1597, 2033, 2098, 2449	Overlapping cars, heavy clutter or reduced visibility.
Diverse or Outlier Configurations	1338, 1461, 250, 1280	Unusual layouts and edge cases used for robustness analysis.

4 Discussion

Overall the system delivers reasonable performance in detecting the 'car' class by combining semantic segmentation with LiDAR geometry. Both visual inspection and the numerical metrics indicate that the approach aligns well in many frames but breaks down under occlusion or when the point density is very sparse. The bounding-box matching logic was inspired by the object-association strategy described in the RWU lecture slides [4].

Would we deploy this in a real car?

No — in its current state the pipeline is not yet robust enough for deployment in a production ADAS system. It lacks real-time inference, dynamic object tracking and full multi-class support. Even so, the implementation is a valuable prototype for prototyping and evaluating sensor-fusion strategies.

References

[1] Ultralytics YOLOv8 GitHub Repository. https://github.com/ultralytics/ultralytics

[2] KITTI-360 Dataset. http://www.cvlibs.net/datasets/kitti-360/

[3] Open3D Documentation. http://www.open3d.org/docs/

[4] Stefan Elser, “Object Tracking and Kalman Filter,” Lidar and Radar Systems Lecture, RWU, Chapter 07, 2024.

[5] Stefan Elser, “Calibration and Fusion,” Lidar and Radar Systems Lecture, RWU, Chapter 08, 2024.

[6] Stefan Elser, “Distance, Velocity, Angle,” Lidar and Radar Systems Lecture, RWU, Chapter 05, 2024.